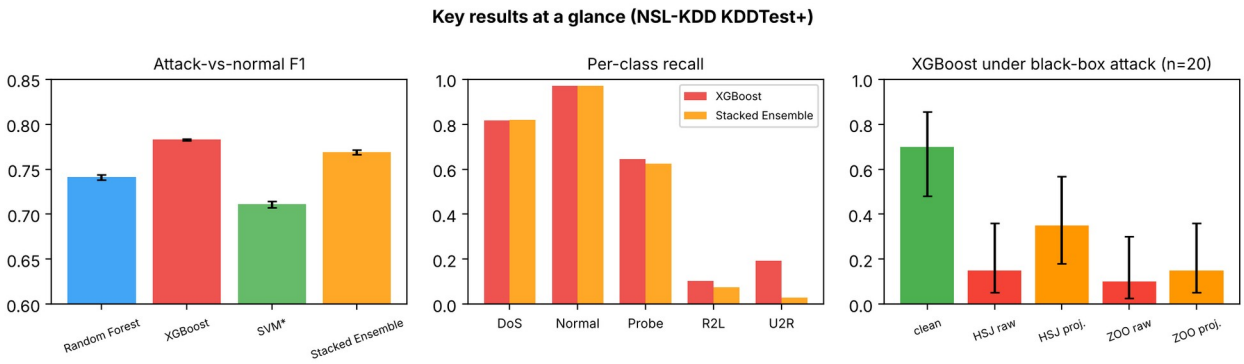
RT-MLIDS: A Real-Time Ensemble Machine Learning Framework for Network Intrusion Detection with Adversarial Robustness Evaluation

Ian Alexander Brighouse Quintana
Department of Cybersecurity and Networks, University of East London, London, United Kingdom
i.brighouse@uel.ac.uk

Version 2 (October 2026). Results were regenerated from the public code (github.com/brigghouse/RT-MLIDS) with a corrected pipeline; they supersede version 1. Main changes: binary and multi-class metrics reported separately, mean ± std over 3 seeds, corrected stacking and label mapping, adversarial samples drawn at random with confidence intervals, and claims narrowed to what the data support.

Abstract—Network intrusion detection systems (IDS) are a core layer of network defence, and machine-learning detectors are increasingly proposed to complement signature-based tools. This paper presents RT-MLIDS, an open-source framework that combines a stacked ensemble of Random Forest and XGBoost (out-of-fold logistic-regression meta-learner), mutual-information feature selection, SMOTE, SHAP explanations, a streaming inference pipeline, and a black-box adversarial evaluation. On the complete NSL-KDD KDDTest+ set (mean over 3 seeds), the ensemble reaches 78.17 ± 0.19% accuracy and 76.89 ± 0.24% F1 for attack-vs-normal detection and 77.00 ± 0.28% five-class accuracy; it does not outperform a single XGBoost model (79.24 ± 0.08% and 77.64 ± 0.40%), which is also ~10× faster. Under black-box HopSkipJump and ZOO attacks against XGBoost (n=20 random test flows), accuracy falls from 70.0% to 15.0% (HopSkipJump, raw) and 10.0% (ZOO, raw); after projecting adversarial points onto valid records the figures are 35.0% and 15.0%. We discuss the limits of these estimates (NSL-KDD age, small attack samples, unconstrained feature dependencies).

Keywords—*machine learning; intrusion detection; network security; random forest; XGBoost; adversarial robustness; ensemble methods; SHAP; real-time detection*



Graphical summary: attack-vs-normal F1 by model (left), per-class recall of the two best models (centre) and accuracy of XGBoost under black-box attack with 95% intervals (right).

I. INTRODUCTION

The global digital economy depends on interconnected network infrastructures spanning cloud platforms, enterprise systems, and critical public services. This dependence has rendered networks a prime target for malicious actors, with global cybercrime costs projected to reach $10.5 trillion annually by 2025 [1]. Network intrusions — including Distributed Denial of Service (DDoS) attacks, port scanning, and Advanced Persistent Threats (APTs) — represent a critical subset of these incidents, capable of compromising sensitive data, disrupting operations, and undermining public trust in digital infrastructure.

Intrusion Detection Systems (IDS) constitute a fundamental defense layer in network security architectures. Conventional IDS solutions, such as Snort and Suricata, operate on a signature-based model wherein network traffic is compared against a database of known attack patterns. While effective against documented threats, this approach is inherently reactive: it cannot detect zero-day exploits, generates high false-positive rates causing analyst alert fatigue, and requires continuous operational maintenance [2].

Machine learning approaches learn discriminative patterns directly from traffic data and can generalize to variants of known attacks. Random Forest [3], XGBoost [4], and deep learning architectures [5] have all been applied to IDS. However, many

studies evaluate offline against passive attackers and report results on easier splits, which limits how far they transfer to deployment.

In response, this paper presents RT-MLIDS (Real-Time Machine Learning Intrusion Detection System), an open-source, reproducible framework. Its contributions are: (1) a stacked ensemble of Random Forest and XGBoost with a meta-learner trained on out-of-fold predictions; (2) an honest evaluation on the full KDDTest+ set reporting binary and multi-class metrics separately, with variation over random seeds; (3) a black-box adversarial evaluation (HopSkipJump and ZOO) that reports results on both raw and validity-projected adversarial points with confidence intervals; and (4) SHAP analysis of the XGBoost base learner. All numbers are regenerated by a single script in the public repository.

II. RELATED WORK

Early ML-based IDS research applied single classifiers to the KDD Cup 1999 and NSL-KDD datasets; Dhanabal and Shantharajah [3] analysed NSL-KDD with standard classifiers, and Yin et al. [5] applied recurrent neural networks to the same benchmark. Reported accuracies on NSL-KDD vary widely with the evaluation protocol: results on random splits or on the easier KDDTest-21 subset are typically much higher than on the full KDDTest+ set, which contains attack types absent from training [13].

Tree-based ensembles and hybrid deep models have been reported to perform strongly on more recent datasets [6], [7], [8]. Nagar et al. [14] review machine-learning approaches to DDoS detection and highlight adversarial robustness and explainability as open challenges, which motivates the evaluation in this paper.

Explainability in IDS has received growing attention; Akintade et al. [9] applied SHAP and LIME to tree-based IDS and identified flow-level features such as destination port and duration as influential.

Adversarial attacks on ML-based IDS have been surveyed by Vitorino et al. [10], who emphasise that realistic attacks must respect the constraints of network traffic, and Yuan et al. [11] studied evasion of deep-learning IDS. Black-box evaluations of tree ensembles on NSL-KDD with explicit feature-validity handling remain less commonly reported; this paper contributes one such evaluation, with the caveats discussed in Section V.

III. PROPOSED FRAMEWORK: RT-MLIDS

A. System Architecture

RT-MLIDS is a six-stage pipeline: flow records, preprocessing (one-hot encoding and mutual-information feature selection), scaling and class balancing, base-learner inference, logistic-regression meta-learning, and alert generation. Fig. 8 illustrates the pipeline. The repository also provides a Kafka-consumer front end for the same preprocessing and inference code; that front end is tested with a mock consumer, and the evaluation in this paper replays NSL-KDD records through it without a broker.

Fig. 8 - RT-MLIDS processing pipeline (offline evaluation; Kafka ingestion is implemented in the repository but not benchmarked)

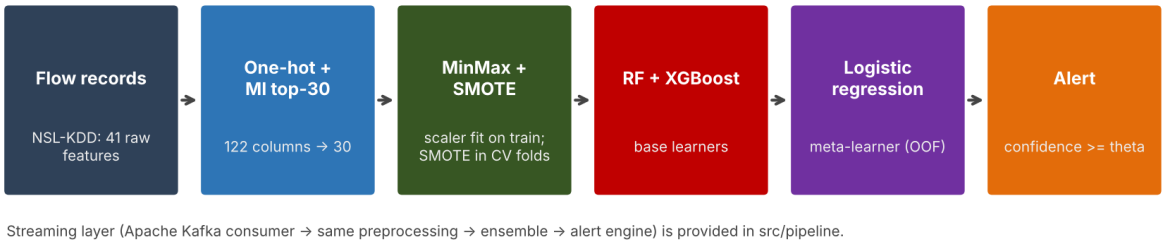


Fig. 8. RT-MLIDS processing pipeline (offline evaluation; the Kafka front end is part of the repository but its transport was not benchmarked).

B. Feature Engineering

Features and selection: NSL-KDD provides 41 pre-extracted features per connection record (the paper's first version incorrectly referred to CICFlowMeter, which is not used). The three categorical features (protocol_type, service, flag) are one-hot encoded, giving 122 columns across KDDTrain+ and KDDTest+. Mutual Information Gain (MIG), fitted on the training set only, retains the top-30 columns:

$$I(X;Y) = \Sigma\ P(x,y)\ \log[P(x,y)\ /\ P(x)P(y)]$$

Class balancing and scaling: The NSL-KDD training set is highly imbalanced (U2R is 0.04% of records). Features are MinMax-scaled (fitted on training data) and SMOTE generates synthetic minority samples by interpolation between neighbours; with the default strategy every class is oversampled to the majority-class size. SMOTE is applied only to training folds and to the final training set, never to evaluation data:

$$x_{new} = x_i + \lambda \cdot (x_{neighbor} - x_i), \quad \lambda \sim Uniform(0,1)$$

C. Ensemble Classification Engine

The classification engine uses stacked generalization. In Layer 1, Random Forest (500 trees, max depth 15) and XGBoost (300 estimators, learning rate 0.05, max depth 6) each produce class-probability vectors. The meta-learner is trained on out-of-fold probabilities from 3-fold stratified cross-validation of the training data, with SMOTE applied inside each training fold, so that meta-features are never computed on data a base learner was fitted on. In Layer 2, multinomial logistic regression combines the concatenated probability vectors:

$$P(c|x) = softmax_c(W \cdot [p_{RF}(x); p_{XGB}(x)] + b)$$

A flow is flagged as an attack when the predicted class is not 'normal' and its confidence is at least $\theta = 0.85$. θ is a fixed operating point, not tuned on the test set; Table II-b reports coverage and accuracy for several thresholds. Fig. 9 illustrates the ensemble architecture.

Fig. 9 - Stacked ensemble: meta-learner trained on out-of-fold probabilities

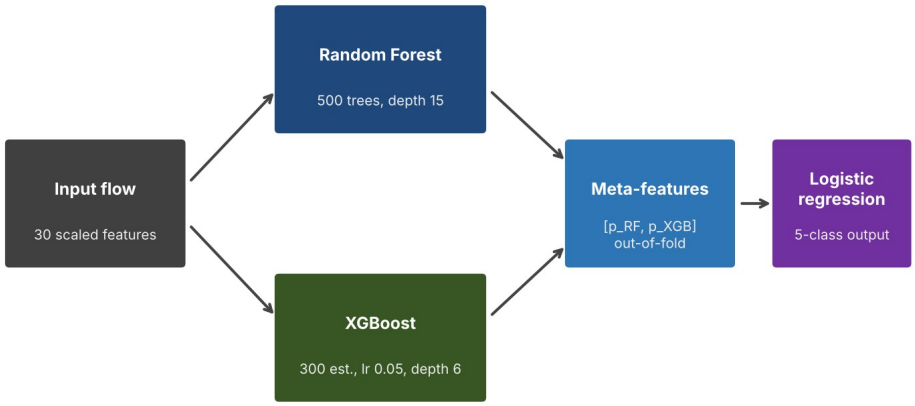


Fig. 9. Stacked ensemble: Random Forest and XGBoost base learners; logistic-regression meta-learner trained on out-of-fold probabilities (3-fold, SMOTE inside each training fold).

D. Real-Time Streaming Pipeline

In the streaming implementation, records are consumed from a Kafka topic (or any iterable), buffered into micro-batches of 512 flows, preprocessed with the fitted pipeline, classified, and passed to an alert engine. We replay 5,000 test records through this code path to measure end-to-end processing cost (Section IV-C); Kafka transport and live feature extraction are not included in that measurement.

IV. EXPERIMENTAL EVALUATION

A. Dataset and Experimental Setup

RT-MLIDS is evaluated on NSL-KDD: 125,973 training and 22,544 test records. Evaluation uses the complete KDDTest+ set, including attack types absent from training, rather than the easier KDDTest-21 subset. All 37 attack names are mapped explicitly to the five categories (an earlier version of the code silently mapped unseen names to U2R). Experiments ran on a 2-vCPU cloud container. Classification results are mean ± standard deviation over 3 random seeds (seeds affect MIG tie-breaking, cross-validation splits, SMOTE and model initialisation; the train/test split is fixed). The SVM is an RBF-kernel SVM trained on a 30,000-record stratified subsample (marked *), because full-data training is impractical.

B. Classification Performance

Table I reports attack-vs-normal (binary) metrics, i.e. the view relevant to an IDS operator; Table I-b reports the five-class weighted metrics, where weighted recall equals accuracy. XGBoost is the best model on both views (binary F1 78.29 ± 0.10%,

five-class accuracy 77.64 ± 0.40%); the stacked ensemble is within about one point (76.89 ± 0.24% and 77.00 ± 0.28%) and does not improve on it. Precision on attack detection is similar for XGBoost and the ensemble (96.71 ± 0.01% vs 96.71 ± 0.02%), so no claim of superior precision is made.

TABLE I

Attack-vs-Normal Metrics — NSL-KDD KDDTest+ (mean ± std over 3 seeds, %)

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	75.38 ± 0.23	92.31 ± 0.27	61.90 ± 0.46	74.11 ± 0.31
XGBoost	79.24 ± 0.08	96.71 ± 0.01	65.76 ± 0.14	78.29 ± 0.10
SVM*	73.08 ± 0.27	91.44 ± 0.05	58.15 ± 0.49	71.09 ± 0.38
Stacked Ensemble	78.17 ± 0.19	96.71 ± 0.02	63.82 ± 0.33	76.89 ± 0.24

TABLE I-b

Model	Accuracy	Precision (w)	Recall (w)	F1 (w)
Random Forest	74.14 ± 0.22	78.68 ± 0.21	74.14 ± 0.22	70.31 ± 0.25
XGBoost	77.64 ± 0.40	82.62 ± 0.41	77.64 ± 0.40	74.35 ± 0.49
SVM*	71.75 ± 0.17	73.19 ± 0.37	71.75 ± 0.17	67.09 ± 0.15
Stacked Ensemble	77.00 ± 0.28	82.46 ± 0.22	77.00 ± 0.28	73.30 ± 0.36

*Five-Class Weighted Metrics — NSL-KDD KDDTest+ (mean ± std over 3 seeds, %; * SVM trained on a 30k subsample)*

The gap to the 99% figures reported elsewhere is expected: the full KDDTest+ set contains attack variants not seen in training, and recall on those variants is low (binary recall about 63–66%). Recall, not precision, is the limiting factor. Per class (Fig. 10), DoS and probe are detected reasonably (recall 82% and 64% for XGBoost) but R2L recall is only 10% (XGBoost) and 8% (ensemble) and U2R 19% and 3%; the headline accuracy hides these rare, high-impact classes.

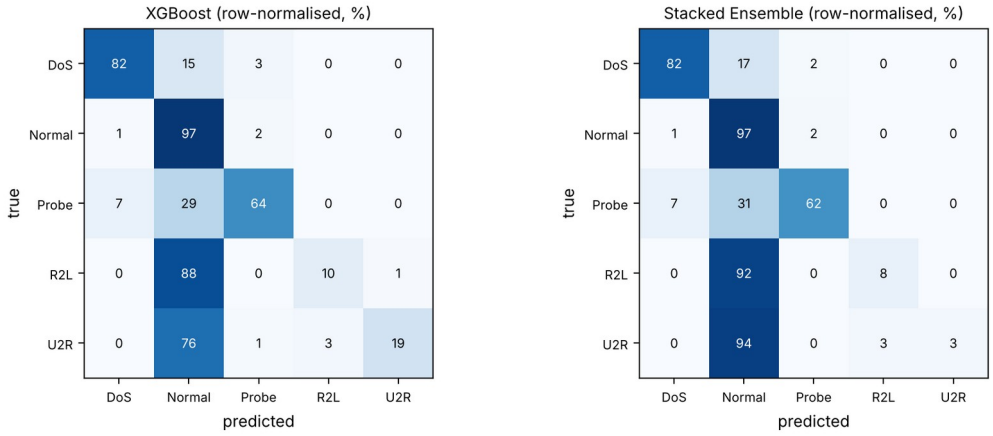


Fig. 10. Row-normalised confusion matrices (seed 0). Rare classes (R2L, U2R) are mostly classified as normal.

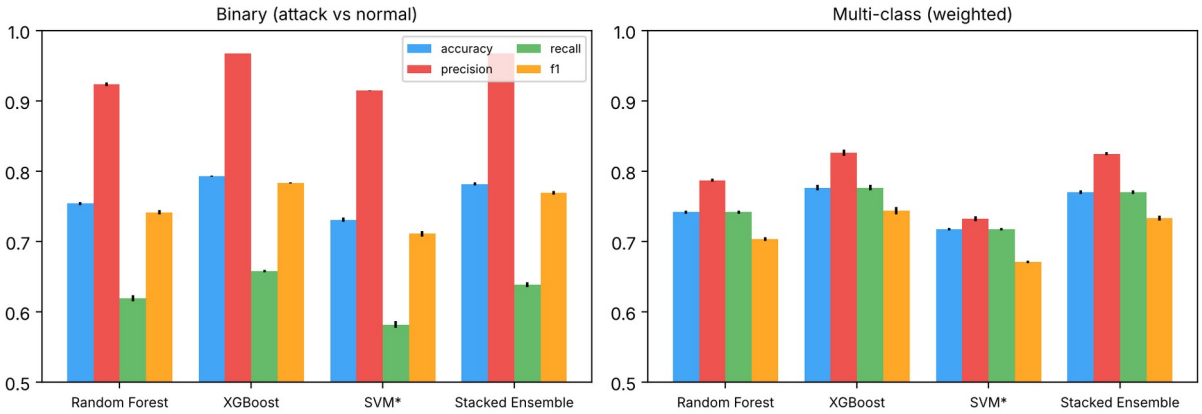


*Fig. 1–2. Classification metrics, mean ± std over seeds: attack-vs-normal (left) and five-class weighted (right). * SVM on a 30k subsample.*

C. Latency and Throughput

Table II reports model-only batch inference cost (batch of 512, single thread, idle machine) and the cost of the end-to-end replay through the streaming code path. XGBoost needs 12.4 µs per flow; the stacked ensemble needs 121.4 µs because it evaluates the 500-tree Random Forest as well. End-to-end replay (preprocessing, inference and alerting, without Kafka transport) costs 150.2 µs per flow for the ensemble. These are single-machine micro-benchmarks, not a deployment latency guarantee.

TABLE II

Inference Latency and Throughput (batch of 512, single thread)

Model	Latency / Flow	Throughput (flows/sec)
Random Forest	102.3 µs	9,778
XGBoost	12.4 µs	80,398
Stacked Ensemble	121.4 µs	8,235
Stacked Ensemble — end-to-end replay	150.2 µs	6,659

Table II-b. Stacked-ensemble confidence threshold (seed 0): θ=0.5: coverage 99.9%, accuracy on covered flows 76.7%; θ=0.7: coverage 98.7%, accuracy on covered flows 77.2%; θ=0.85: coverage 97.1%, accuracy on covered flows 77.7%; θ=0.95: coverage 95.2%, accuracy on covered flows 78.4%.

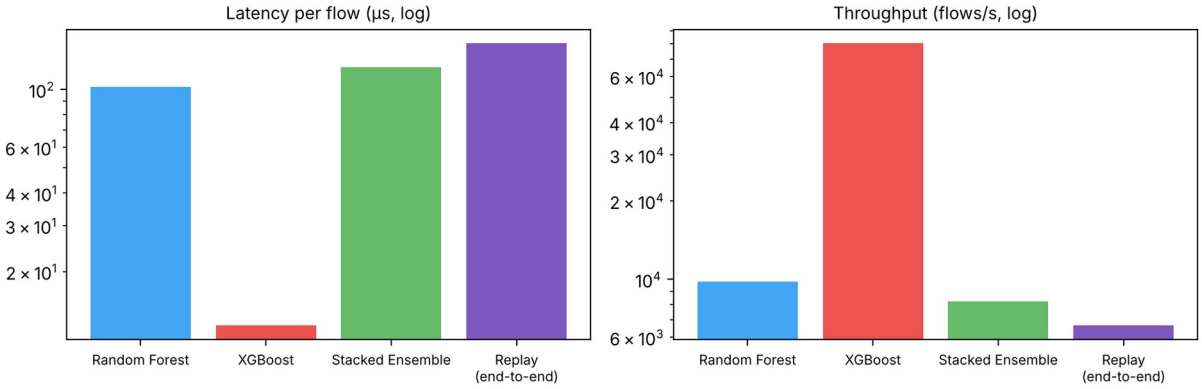


Fig. 3–4. Latency per flow and throughput (log scale), model-only inference and end-to-end replay, single thread.

D. Adversarial Robustness

Adversarial attacks were run through ART's black-box wrapper (predicted probabilities only) on a seeded random sample of KDDTest+. Because a raw adversarial vector may not be a valid NSL-KDD record (values outside the scaled range, fractional one-hot columns), accuracy is reported on the raw points and after projection onto valid records (clipping to the scaled range and

rounding one-hot columns). Against XGBoost (n=20, clean accuracy 70.0% [48–85]), HopSkipJump reduces five-class accuracy to 15.0% [5–36] raw and 35.0% [18–57] projected; ZOO reduces it to 10.0% [3–30] raw and 15.0% [5–36] projected. Query budgets were limited (XGBoost: HopSkipJump 20 iterations, ZOO 100 iterations; ensemble: HopSkipJump 5 iterations, ZOO 20 iterations). Drops are stated relative to the clean accuracy of the same sample. The stacked ensemble was attacked with a much smaller query budget (n=20; each query evaluates the full ensemble), so its rows are indicative only.

TABLE III

Adversarial Robustness — five-class accuracy, 95% Wilson interval in brackets

Condition	Accuracy [95% CI]	n
XGBoost — clean	70.0% [48–85]	20
XGBoost — HopSkipJump, raw	15.0% [5–36]	20
XGBoost — HopSkipJump, projected	35.0% [18–57]	20
XGBoost — ZOO, raw	10.0% [3–30]	20
XGBoost — ZOO, projected	15.0% [5–36]	20
Stacked Ensemble — clean	70.0% [48–85]	20
Stacked Ensemble — HopSkipJump, raw	5.0% [1–24]	20
Stacked Ensemble — HopSkipJump, projected	35.0% [18–57]	20
Stacked Ensemble — ZOO, raw	40.0% [22–61]	20
Stacked Ensemble — ZOO, projected	45.0% [26–66]	20

These results show that a boundary-search black-box attack can substantially reduce this model's accuracy on a small sample, and that the projection to valid records changes the conclusion materially. They do not establish a general robustness figure: the attacks do not enforce the dependencies between NSL-KDD features (e.g. rate features derived from counts), the samples are small, and intervals are wide.

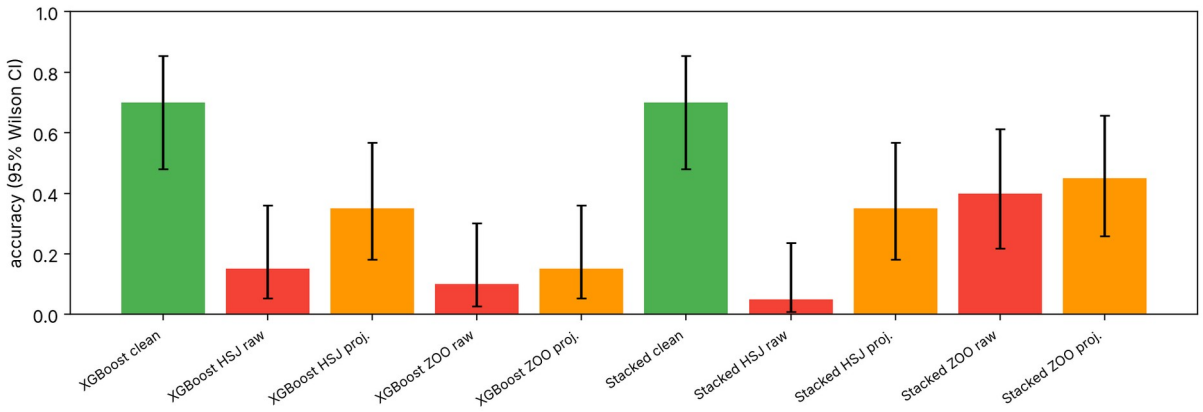


Fig. 5–6. Accuracy under black-box attack (raw and projected-to-valid adversarial points), with 95% Wilson intervals. Samples are small; see Section V.

E. SHAP Interpretability Analysis

Table IV lists the five features with the highest mean absolute SHAP value for the XGBoost base learner (500 test flows, averaged over classes; seed 0). src_bytes ranks first, followed by connection-count and host-rate features, which is consistent with volume- and frequency-based DoS and probe signatures. SHAP describes model behaviour, not causal importance; the one-hot 'service' columns are split across many columns and are therefore under-represented in a per-column ranking.

TABLE IV

Top-5 Features by Mean Absolute SHAP Value (XGBoost)

Rank	Feature	Mean |SHAP|
1	src_bytes	1.1989
2	dst_host_srv_count	0.8850
3	count	0.7533
4	dst_bytes	0.7352
5	dst_host_same_src_port_rate	0.5007

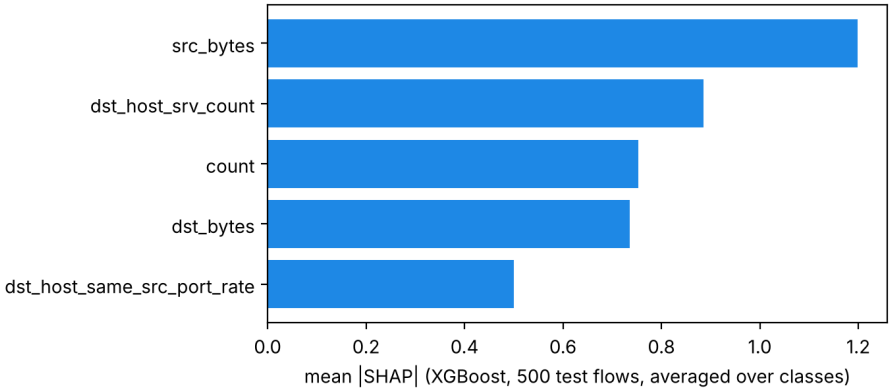


Fig. 7. Top-5 features by mean absolute SHAP value for the XGBoost base learner.

V. DISCUSSION

On the full KDDTest+ benchmark the stacked ensemble did not outperform a single tuned XGBoost model, and it is about 10× more expensive at inference. The practical recommendation from these data is therefore to use XGBoost alone unless a different benchmark shows a benefit from stacking. Reported results depend on the evaluation view: binary attack-vs-normal metrics are higher than five-class metrics, and the two should not be mixed.

The adversarial results indicate sensitivity to black-box boundary-search attacks, but the magnitude depends strongly on whether adversarial points are required to be valid records. A realistic attacker must craft real flows whose features are mutually consistent, which is harder than the vector-space attacks used here; conversely, our projection step does not enforce all such constraints, so the projected figures are an upper bound on attacker success. Adversarial training and perturbation detection remain future work.

Limitations: (1) NSL-KDD reflects 1999-era traffic and a fixed train/test split, so seeds capture training variability but not dataset variability; (2) per-class U2R and R2L results are noisy given their sizes; (3) adversarial samples are small (n=20 for XGBoost, n=20 for the ensemble) with a reduced query budget for the ensemble; (4) latency was measured on a 2-vCPU container, single-threaded, and excludes Kafka transport and feature extraction; (5) no comparison with other published IDS models was run under this protocol.

Explainability: the SHAP ranking is plausible given domain knowledge of DoS and probe traffic, but it is model-specific and should be read together with the validity caveats above.

VI. CONCLUSION

This paper presented RT-MLIDS, an open, reproducible ensemble IDS framework evaluated on the complete NSL-KDD KDDTest+ set. The stacked ensemble reaches 76.89 ± 0.24% binary F1 and 77.00 ± 0.28% five-class accuracy but does not beat a single XGBoost model. Black-box attacks reduce XGBoost's accuracy on a random sample, with effect sizes that depend on whether adversarial points are projected to valid records. The full code, results file and tests are available in the public repository.

Future work: (1) adversarial training and feature-constrained attacks; (2) evaluation on modern datasets such as CIC-IDS-2018 and on encrypted traffic; (3) end-to-end latency measurement with a live Kafka deployment and a flow extractor.

. ACKNOWLEDGMENT

The author thanks the Department of Cybersecurity and Networks at the University of East London for academic support. The NSL-KDD dataset is provided by the Canadian Institute for Cybersecurity [13].

. REFERENCES

[1] Cybersecurity Ventures, "Cybercrime To Cost The World $10.5 Trillion Annually By 2025," 2020.

[2] M. Roesch, "Snort: Lightweight intrusion detection for networks," in Proc. USENIX LISA, 1999, pp. 229–238.

[3] L. Dhanabal and S. Shantharajah, "A study on NSL-KDD dataset for intrusion detection system," Int. J. Adv. Res. Comput. Commun. Eng., vol. 4, no. 6, pp. 446–452, 2015.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. ACM SIGKDD, 2016, pp. 785–794.

[5] C. Yin, Y. Zhu, J. Fei, and X. He, "A deep learning approach for intrusion detection using recurrent neural networks," IEEE Access, vol. 5, pp. 21954–21961, 2017.

[6] D. Chou and M. Yeh, "Toward developing network intrusion detection system using adaptive random forest," in Proc. IEEE CSE, 2019, pp. 1–7.

[7] F. A. Khan, A. Gumaei, A. Derhab, and A. Hussain, "A novel two-stage deep learning model for efficient network intrusion detection," IEEE Access, vol. 7, pp. 30373–30385, 2019.

[8] M. Ibrahim and A. Al-Wadi, "Enhancing IoMT network security using ensemble learning-based IDS," J. Eng. Research, 2024.

[9] S. Akintade, S. Kim, and K. Roy, "Explaining ML-based feature selection of IDS for IoT and CPS devices," in Proc. ICMLA, 2023.

[10] J. Vitorino, I. Praça, and E. Maia, "SoK: Realistic adversarial attacks and defenses for intelligent network intrusion detection," Comput. Secur., vol. 134, 2023.

[11] X. Yuan et al., "A simple framework to enhance adversarial robustness of deep learning-based IDS," Comput. Secur., vol. 137, 2024.

[12] NIST, "Guide to Intrusion Detection and Prevention Systems (IDPS)," Special Publication 800-94, 2007.

[13] M. Tavallaee, E. Bagheri, W. Lu, and A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proc. IEEE CISDA, 2009.

[14] A. Nagar, S. Tatreh, and B. K. Pandey, "A survey on machine learning based approaches for detecting and preventing distributed DoS attacks in IoT," J. Inf. Syst. Eng. Manag., vol. 9, no. 4s, 2024.